Lekcja 16 - Kryptografia w praktyce. Część 2

1. **Przygotowanie**

```
┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ gpg --version
gpg (GnuPG) 2.4.9
libgcrypt 1.12.2
Copyright (C) 2025 g10 Code GmbH
License GNU GPL-3.0-or-later <https://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Home: /home/kali/.gnupg
Supported algorithms:
Pubkey: RSA, ELG, DSA, ECDH, ECDSA, EDDSA
Cipher: IDEA, 3DES, CAST5, BLOWFISH, AES, AES192, AES256, TWOFISH,
        CAMELLIA128, CAMELLIA192, CAMELLIA256
Hash: SHA1, RIPEMD160, SHA256, SHA384, SHA512, SHA224
Compression: Uncompressed, ZIP, ZLIB, BZIP2

┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ LAB_DIR="$(mktemp -d "$PWD/learnit-gpg.XXXXXX")"

┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ mkdir -p "$LAB_DIR/gpg" "$LAB_DIR/outbox" "$LAB_DIR/inbox"

┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ chmod 700 "$LAB_DIR/gpg"

┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ export GNUPGHOME="$LAB_DIR/gpg"

┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ export GPG_TTY="$(tty)"

┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ printf "Moj katalog: %s\n" "$LAB_DIR"
```

Osoba A wykonuje:

```
┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ MY_NAME='Alicja Lab'

┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ MY_EMAIL='alicja@lab.invalid'

┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ PARTNER_EMMAIL='bob@lab.invalid'
```

Osoba B wykonuje:

```
┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ MY_NAME='Bob Lab'

┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ MY_EMAIL='bob@lab.invalid'

┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ PARTNER_EMMAIL='alicja@lab.invalid'
```

Tworzenie własnego klucza do podpisów:

```
┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ gpg --quick-generate-key "$MY_NAME <$MY_EMAIL>" ed25519 sign 1y
gpg: keybox '/home/kali/lab 16 gpg/learnit-gpg.jbwM7X/gpg/pubring.kbx' created
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: /home/kali/lab 16 gpg/learnit-gpg.jbwM7X/gpg/trustdb.gpg: trustdb created
gpg: directory '/home/kali/lab 16 gpg/learnit-gpg.jbwM7X/gpg/openpgp-revocs.d' created
gpg: revocation certificate stored as '/home/kali/lab 16 gpg/learnit-gpg.jbwM7X/gpg/openpgp-revocs.d/344F1EDC8F0934ECEDE912C3B713F8F891486247.rev'
public and secret key created and signed.

pub   ed25519 2026-10-14 [SC] [expires: 2027-10-14]
      344F1EDC8F0934ECEDE912C3B713F8F891486247
uid                      Bob Lab <bob@lab.invalid>


┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ gpg --fingerprint "$MY_EMAIL"
gpg: checking the trustdb
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: depth: 0  valid:   1  signed:   0  trust: 0-, 0q, 0n, 0m, 0f, 1u
gpg: next trustdb check due at 2027-10-14
pub   ed25519 2026-10-14 [SC] [expires: 2027-10-14]
      344F 1EDC 8F09 34EC EDE9  12C3 B713 F8F8 9148 6247
uid           [ultimate] Bob Lab <bob@lab.invalid>


┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ gpg --armor --export "$MY_EMAIL" > "$LAB_DIR/outbox/public.asc"
```

2. Osoba B sprawdza klucz i podpis

```
┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ gpg --import "$LAB_DIR/inbox/public.asc"
gpg: key 33161B8E38B31A3A: public key "Alicja Lab <alicja@lab.invalid>" imported
gpg: Total number processed: 1
gpg:               imported: 1

┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ gpg --fingerprint "$PARTNER_EMAIL"
gpg: error reading key: Invalid user ID

┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ PARTNER_EMAIL='alicja@lab.invalid'


┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ gpg --fingerprint "$PARTNER_EMAIL"
pub   ed25519 2026-10-08 [SC] [expires: 2027-10-08]
      C2C2 7FB0 1672 43A7 A787  BEAD 3316 1B8E 38B3 1A3A
uid           [ unknown] Alicja Lab <alicja@lab.invalid>


┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ gpg --verify "$LAB_DIR/inbox/raport.txt.sig.asc" "$LAB_DIR/inbox/raport.txt"
gpg: Signature made Thu 08 Oct 2026 01:06:43 PM EDT
gpg:                using EDDSA key C2C27FB0167243A7A787BEAD33161B8E38B31A3A
gpg:                issuer "alicja@lab.invalid"
gpg: Good signature from "Alicja Lab <alicja@lab.invalid>" [unknown]
gpg: WARNING: The key's User ID is not certified with a trusted signature!
gpg:          There is no indication that the signature belongs to the owner.
Primary key fingerprint: C2C2 7FB0 1672 43A7 A787  BEAD 3316 1B8E 38B3 1A3A
```

Ze względu na literówkę w PARTNER*_EMAIL, po wprowadzeniu: gpg - -fingerprint "$PARTNER_EMAIL", otrzymałem błąd:*

```
┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ gpg --fingerprint "$PARTNER_EMAIL"
gpg: error reading key: Invalid user ID

┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ PARTNER_EMAIL='alicja@lab.invalid'


┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ gpg --fingerprint "$PARTNER_EMAIL"
pub   ed25519 2026-10-08 [SC] [expires: 2027-10-08]
      C2C2 7FB0 1672 43A7 A787  BEAD 3316 1B8E 38B3 1A3A
uid           [ unknown] Alicja Lab <alicja@lab.invalid>
```

*więc wprowadziłem zmienną PARTNER*_EMAIL ponownie i dopiero wtedy sprawdzenie zakończyło się sukcesem.

3. **Osoba B zmienia kopię dokumentu**

```
┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ sed 's/Zatwierdzam/Nie zatwierdzam/' "$LAB_DIR/inbox/raport.txt" > "$LAB_DIR/inbox/raport-zmieniony.txt"

┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ gpg --verify "$LAB_DIR/inbox/raport.txt.sig.asc" "$LAB_DIR/inbox/raport-zmieniony.txt"
gpg: Signature made Thu 08 Oct 2026 01:06:43 PM EDT
gpg:                using EDDSA key C2C27FB0167243A7A787BEAD33161B8E38B31A3A
gpg:                issuer "alicja@lab.invalid"
gpg: BAD signature from "Alicja Lab <alicja@lab.invalid>" [unknown]
```

Teraz B sprawdza ten sam podpis z oryginalnym dokumentem:

```
┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ gpg --verify "$LAB_DIR/inbox/raport.txt.sig.asc" "$LAB_DIR/inbox/raport.txt"
gpg: Signature made Thu 08 Oct 2026 01:06:43 PM EDT
gpg:                using EDDSA key C2C27FB0167243A7A787BEAD33161B8E38B31A3A
gpg:                issuer "alicja@lab.invalid"
gpg: Good signature from "Alicja Lab <alicja@lab.invalid>" [unknown]
gpg: WARNING: The key's User ID is not certified with a trusted signature!
gpg:          There is no indication that the signature belongs to the owner.
Primary key fingerprint: C2C2 7FB0 1672 43A7 A787  BEAD 3316 1B8E 38B3 1A3A
```

Zamiana ról. Osoba A podpisuje dokument.

A jest autorem, a B weryfikatorem. A wykonuje:

```
┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ cat > "$LAB_DIR/outbox/raport.txt" <<EOF
Autor: $MY_NAME
Cwiczenie: weryfikacja podpisow GPG
Zatwierdzam wersje testowa raportu
EOF

┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ cat $LAB_DIR/outbox/raport.txt
Autor: Bob Lab
Cwiczenie: weryfikacja podpisow GPG
Zatwierdzam wersje testowa raportu

┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ gpg --armor --local-user "$MY_EMAIL" --output "$LAB_DIR/outbox/raport.txt.sig.asc" --detach-sign "$LAB_DIR/outbox/raport.txt"

┌──(kali㉿kali)-[~/lab 16 gpg]
└─$ ls -l "$LAB_DIR/outbox"
total 12
-rw-rw-r-- 1 kali kali 396 Oct 14 16:39 public.asc
-rw-rw-r-- 1 kali kali  86 Oct 14 17:30 raport.txt
-rw-rw-r-- 1 kali kali 252 Oct 14 17:32 raport.txt.sig.asc
```

Dokonałem sprawdzenia zawartości folderu /outbox

Public.asc - istnieje w /outboxmój klucz publiczny
Raport.txt - istnieje w /outbox oryginalna treść
Raport.txt.sig.asc - istnieje w /outbox

Test	**Wynik GnuPG lub obserwacja**	**Co z tego wynika?**
Sprawdzenie przed importem klucza autora	Odczytaliśmy na głos fingerprint, porównując ze sobą zawarte informacje.	Bez klucza publicznego nie można zweryfikować podpisu.
Oryginał i właściwy klucz publiczny	Good signature	Podpis jest poprawny, a dokument nie został zmieniony.

Zmieniony dokument, ten sam podpis	Bad signature	Zmiana treści powoduje niezgodność podpisu.
Ponowna weryfikacja oryginału	Good signature	Oryginalny dokument nadal ma poprawny podpis.
Porównanie pełnego odcisku klucza partnera	Porównanie odcisku klucz z autorem/partnerem.	Trzeba potwierdzić zgodność z odciskiem podanym przez A inną drogą.

Dlaczego Good signature samo w sobie nie potwierdza tożsamości realnej osoby?

Ponieważ każdy może utworzyć klucz z dowolnym imieniem i adresem e-mail. Poprawny podpis nie oznacza automatycznie, że klucz należy do wskazanej osoby.

Czym różni się No public key od BAD signature?

No public key oznacza brak klucza potrzebnego do sprawdzenia podpisu. BAD signature oznacza, że podpis nie pasuje do sprawdzanego dokumentu przy użyciu danego klucza.

Co zrobicie, jeśli pełny odcisk otrzymanego klucza różni się od odcisku potwierdzonego przez partnera?

Przerywamy weryfikację i nie ufamy otrzymanemu kluczowi. Kontaktujemy się z partnerem, aby wyjaśnić niezgodność.

Co sprawdza Good signature?

Potwierdza, że podpis pasuje do dokumentu i został utworzony kluczem prywatnym odpowiadającym użytemu kluczowi publicznemu.

Skąd wiemy, że użyliśmy klucza, którego odcisk podała osoba A?

Sprawdzamy pełny odcisk otrzymanego klucza poleceniem gpg --fingerprint i porównujemy go z odciskiem przekazanym przez drugą osobę inną drogą.

Dlaczego ten sam podpis daje dwa różne wyniki?

Ponieważ podpis dotyczy konkretnej treści dokumentu. Dla oryginału otrzymujemy "Good signature", a po zmianie jednego słowa "BAD signature".

Czy podpis ukrywa treść raport.txt?

Podpis cyfrowy służy do sprawdzania autentyczności i integralności dokumentu, a nie do szyfrowania. Treść pliku raport.txt pozostaje czytelna.